



Stamp It! All Programs Must Report Their Version

published 2026-04-05 in tags [golang](#) [nix](#)

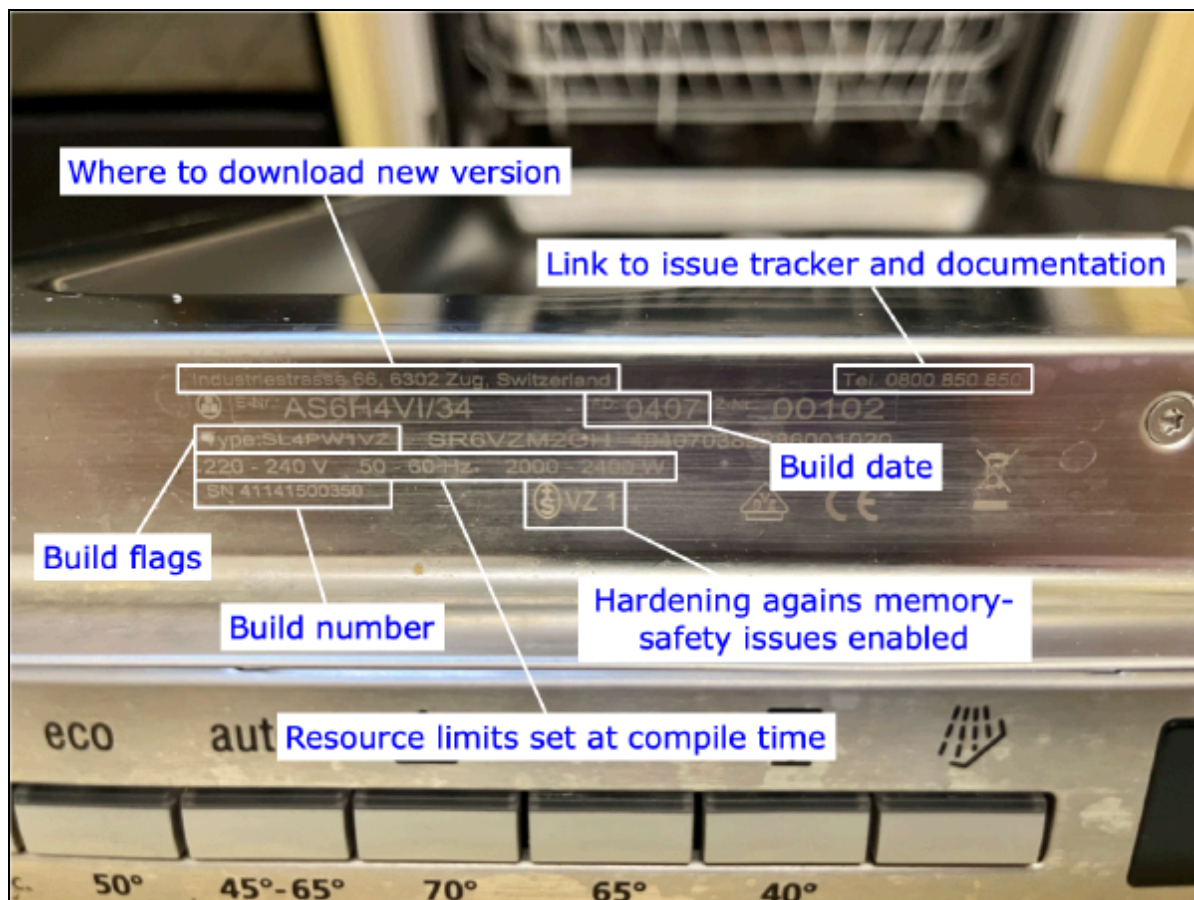
Recently, during a production incident response, I guessed the root cause of an outage correctly within less than an hour (cool!) and submitted a fix just to rule it out, only to then spend many hours fumbling in the dark because we lacked visibility into version numbers and rollouts... 😞

This experience made me think about software versioning again, or more specifically about build info (build versioning, version stamping, however you want to call it) and version reporting. I realized that for the i3 window manager, I had solved this problem well over a decade ago, so it was really unexpected that the problem was decidedly not solved at work.

In this article, I'll explain how 3 simple steps (Stamp it! Plumb it! Report it!) are sufficient to save you hours of delays and stress during incident response.

Why are our versioning standards so low?!

Every household appliance has incredibly detailed versioning! Consider this dishwasher:



(Thank you Feuermurmél for sending me this lovely example!)

I observed a couple household appliance repairs and am under the impression that if a repair person cannot identify the appliance, they would most likely refuse to even touch it.

So why are our standards so low in computers, in comparison? Sure, consumer products are typically versioned *somehow* and that's typically good enough (except for, say, USB 3.2 Gen 1×2!). But recently, I have encountered too many developer builds that were not adequately versioned!

Software Versioning

Unlike a physical household appliance with a stamped metal plate, software is constantly updated and runs in places and structures we often cannot even see.

Let's dig into what we need to increase our versioning standard!

Usually, software has a **name** and some **version number** of varying granularity:

- Chrome
- Chrome 146
- Chrome 146.0.7680.80
- Chrome f08938029c887ea624da7a1717059788ed95034d-refs/branch-heads/7680_65@{#34}

All of these identify the Chrome browser on my computer, but each at different granularity.

All are correct and useful, depending on the context. Here's an example for each:

1. "This works in Chrome for me, did you test in Firefox?"
2. "Chrome 146 contains broken middle-click-to-paste-and-navigate"
3. "I run Chrome 146.0.7680.80 and cannot reproduce your issue"
4. "Apply this patch on top of Chrome f08938029c887ea624da7a1717059788ed95034d-refs/branch-heads/7680_65@{#34} and follow these steps to reproduce: [...]"

After creating the [i3 window manager](#), I quickly learned that for user support, it is very valuable for programs to clearly identify themselves. Let me illustrate with the following case study.

Case Study: i3's `--version` and `--moreversion`

When running `i3 --version`, you will see output like this:

```
% i3 --version
i3 version 4.24 (2024-11-06) © 2009 Michael Stapelberg and contributors
```

Each word was carefully deliberated and placed. Let me dissect:

1. `i3 version 4.24`: I could have shortened this to `i3 4.24` or maybe `i3 v4.24`, but I figured it would be helpful to be explicit because `i3` is such a short name. Users might mumble aloud “What’s an i-3-4-2-4?”, but when putting “version” in there, the implication is that `i3` is some computer thing (→ a computer program) that exists in version 4.24.
2. `(2024-11-06)` is the release date so that you can immediately tell if “4.24” is recent.
3. `© 2009 Michael Stapelberg` signals when the project was started and who is the main person behind it.
4. `and contributors` gives credit to the many people who helped. `i3` was never a one-person project; it was always a group effort.

When doing user support, there are a couple of questions that are conceptually easy to ask the affected user and produce very valuable answers for the developer:

1. Question: “Which version of `i3` are you using?”
 - Since `i3` is not a typical program that runs in a window (but a window manager / desktop environment), there is no Help → About menu option.
 - Instead, we started asking: What is the output of `i3 --version`?
2. Question: “*Are you reporting a new issue or a preexisting issue? To confirm, can you try going back to the version of `i3` you used previously?*”. The technical terms for “going back” are downgrade, rollback or revert.
 - Depending on the Linux distribution, this is either trivial or a nightmare.
 - With NixOS, it’s trivial: you just boot into an older system “generation” by selecting that version in the bootloader. Or you revert in git, if your configs are version-controlled.
 - With imperative Linux distributions like Debian Linux or Arch Linux, if you did not take a file system-level snapshot, there is no easy and reliable way to go back after upgrading your system. If you are lucky, you can just `apt install` the older version of `i3`. But you might run into dependency conflicts (“version hell”).
 - I know that it is *possible* to run older versions of Debian using snapshot.debian.org, but it is just not very practical, at least when I last tried.
3. Can you check if the issue is still present in the latest `i3` development version?
 - Of course, I could also try reproducing the user issue with the latest release version, and **then one additional time** on the latest development version.
 - But this way, the verification step moves to the affected user, which is good because it filters for highly-motivated bug reporters (higher chance the bug report actually results in

a fix!) and it makes the user reproduce the bug *twice*, figuring out if it's a flaky issue, hard-to-reproduce, if the reproduction instructions are correct, etc.

- A natural follow-up question: “*Does this code change make the issue go away?*” This is easy to test for the affected user who now has a development environment.

Based on my experiences with asking these questions many times, I noticed a few patterns in how these debugging sessions went. In response, I introduced another way for i3 to report its version in i3 v4.3 (released in September 2012): a `--moreversion` flag! Now I could ask users a small variation of the first question: What is the output of `i3 --moreversion`? Note how this also transfers well over spoken word, for example at a computer meetup:

Michael: Which version are you using?

User: How can I check?

Michael: Run this command: `i3 --version`

User: It says 4.24.

Michael: Good, that is recent enough to include the bug fix. Now, we need more version info! Run `i3 --moreversion` please and tell me what you see.

When you run `i3 --moreversion`, it does not just report the version of the i3 program you called, it also connects to the running i3 window manager process in your X11 session using [its IPC \(interprocess communication\) interface](#) and reports the running i3 process's version, alongside other key details that are helpful to show the user, like which configuration file is loaded and when it was last changed:

```
% i3 --moreversion
Binary i3 version:  4.24 (2024-11-06) © 2009 Michael Stapelberg and...
Running i3 version: 4.24 (2024-11-06) (pid 2521)
Loaded i3 config:
  /home/michael/.config/i3/config (main)
  (last modified: 2026-03-15T23:09:27 CET, 1101585 seconds ago)
```

```
The i3 binary you just called:
/nix/store/0zn9r4263fjppqah6vdzla1fn0ahp8xc2-i3-4.24/bin/i3
The i3 binary you are running: i3
```

This might look like a lot of detail on first glance, but let me spell out why this output is such a valuable debugging tool:

1. Connecting to i3 via the IPC interface is an interesting test in and of itself. If a user sees `i3 --moreversion` output, that implies they will also be able to run debugging commands like (for example) `i3-msg -t get_tree > /tmp/tree.json` to capture the full layout state.
2. During a debugging session, running `i3 --moreversion` is an easy check to see if the version you just built is actually effective (see the `Running i3 version` line).

- Note that this is the same check that is relevant during production incidents: verifying that *effectively running* matches *supposed to be running* versions.

3. Showing the full path to the loaded config file will make it obvious if the user has been editing the wrong file. If the path alone is not sufficient, the modification time (displayed both absolute and relative) will flag editing the wrong file.

I use NixOS, BTW, so I automatically get a stable identifier (0zn9r4263fjppqah6vdz1alfn0ahp8xc2-i3-4.24) for *the specific build* of i3.

```
% ls -l $(which i3)
lrwxrwxrwx 1 root root 58 1970-01-01 01:00 /run/current-system/sw/bin/i3
-> /nix/store/0zn9r4263fjppqah6vdz1alfn0ahp8xc2-i3-4.24/bin/i3
```

To see the build recipe (“derivation” in Nix terminology) which produced this Nix store output (0zn9r4263...-i3-4.24), I can run `nix derivation show`:

```
% nix derivation show /nix/store/0zn9r4263fjppqah6vdz1alfn0ahp8xc2-i3-4.24
{
  "/nix/store/z7ly4kvgixf29rlz01ji4nywbajfifk4-i3-4.24.drv": {
    [...]
  }
}
```

► Click here to expand the full `nix derivation show` output if you are curious

Unfortunately, I am not aware of a way to go from the derivation to the `.nix` source, but at least one can check that a certain source results in an identical derivation.

Developer builds

The versioning I have described so far is sufficient for most users, who will not be interested in tracking intermediate versions of software, but only the released versions.

But what about developers, or any kind of user who needs more precision?

When building i3 from git, it reports the git revision it was built from, using [git-describe\(1\)](#) :

```
~/i3/build % git describe
4.25-23-g98f23f54
~/i3/build % ninja
[110/110] Linking target i3
~/i3/build % ./i3 --version
i3 version 4.25-23-g98f23f54 © 2009 Michael Stapelberg and contributors
```

A modified working copy gets represented by a `+` after the revision:

```
~/i3/build % echo '// dirty working copy' >> ../src/main.c && ninja
[104/104] Linking target i3bar
```

Reporting the git revision (or VCS revision, generally speaking) is the most useful choice.

This way, we catch the following common mistakes:

- People build from the wrong revision.
- People build, but forget to install.
- People install, but their session does not pick it up (wrong location?).

Most Useful: Stamp The VCS Revision

As we have seen above, the single most useful piece of version information is the VCS revision. We can fetch all other details (version numbers, dates, authors, ...) from the VCS repository.

Now, let's demonstrate the best case scenario by looking at how Go does it!

Go always stamps! 🎉

Go has become my favorite programming language over the years, in big part because of the good taste and style of the Go developers, and of course also because of the high-quality tooling:

Why Go is my favorite programming language

I strive to respect everybody's personal preferences, so I usually steer clear of debates about which is the best programming language, text editor or operating system.

However, recently I was asked a couple of times why I like and use a lot of Go, so here is a coherent article to fill in the blanks of my ad-hoc in-person ramblings :-). Read more →

Therefore, I am pleased to say that Go implements the gold standard with regard to software versioning: it stamps VCS buildinfo by default! 🎉 This was introduced in [Go 1.18 \(March 2022\)](#):

Additionally, the `go` command embeds information about the build, including build and tool tags (set with `-tags`), compiler, assembler, and linker flags (like `-gcflags`), whether `cgo` was enabled, and if it was, the values of the `cgo` environment variables (like `CGO_CFLAGS`).

Both VCS and build information may be read together with module information using `go version -m file` or [runtime/debug.ReadBuildInfo](#) (for the currently running binary) or the new [debug/buildinfo](#) package.

❗ **Note:** Before Go 1.18, the standard approach was to use `-ldflags -X main.version=$(git describe)` or similar explicit injection. This setup works (and can still be seen in many places) but requires making changes to the application code, whereas the Go 1.18+ stamping requires no extra steps.

What does this mean in practice? Here is a diagram for the common case: building from git:

.

This covers most of my hobby projects!

Many tools I just `go install`, or `CGO_ENABLED=0 go install` if I want to easily copy them around to other computers. Although, I am managing more and more of my software in NixOS.

When I find a program that is not yet fully managed, I can use `gops` and the `go` tool to identify it:

```
root@ax52 ~ % nix run nixpkgs#gops
2573594 1      dcs-package-importer  go1.26.1 /nix/store/clby54zb003ibai8j70pwad629lhqfly-
dcs-unstable/bin/dcs-package-importer
2573576 1      dcs-source-backend   go1.26.1 /nix/store/clby54zb003ibai8j70pwad629lhqfly-
dcs-unstable/bin/dcs-source-backend
2573566 1      debiman              go1.25.5 /srv/man/bin/debiman
[...]
root@ax52 ~ % nix run nixpkgs#go -- version -m /srv/man/bin/debiman
/srv/man/bin/debiman: go1.25.5
  path  github.com/Debian/debiman/cmd/debiman
  mod   github.com/Debian/debiman          v0.0.0-20251230101540-ac8f5391b43b+dirty
  [...]
  dep   paul.t.ag/go/debian              v0.18.0 h1:nr0iyy0U5QLG1VPnhZLNhnCcHx58kukvBJp+dvaM6CQ=
  dep   paul.t.ag/go/topsort              v0.1.1  h1:L0QnhUly6LmTv0e3DEzbN2q6/FGgAcQvaEw65S53Bg4=
  build -buildmode=exe
  build -compiler=gc
  build
DefaultGODEBUG=containermaxprocs=0,decoratemappings=0,tlssha1=1,updatemaxprocs=0,x509sha256sk
id=0
  build CGO_ENABLED=0
  build GOARCH=amd64
  build GOOS=linux
  build GOAMD64=v1
  build vcs=git
  build vcs.revision=ac8f5391b43bc1a9dbdc99f6179e2fb7d7414a04
  build vcs.time=2025-12-30T10:15:40Z
  build vcs.modified=true
root@ax52 ~ %
```

It's very cool that Go does the right thing by default!

Systems that consist of 100% Go software (like my [gokrazy Go appliance platform](#)) are fully stamped! For example, the gokrazy web interface shows me exactly which version and dependencies went into the gokrazy/rsync build on my [scan2drive appliance](#).

Despite being fully stamped, note that gokrazy only shows the module versions, and no VCS buildinfo, because it currently suffers from the same gap as Nix:

Go Version Reporting

For the gokrazy packer, which follows a rolling release model (no version numbers), I ended up with a few lines of Go code (see below) to display a git revision, no matter if you installed the packer from a Go module or from a git working copy.

The code either displays `vcs.revision` (the easy case; built from git) or extracts the revision from the Go module version of the main module ([BuildInfo.Main.Version](#)):

What are the other cases? These examples illustrate the scenarios I usually deal with:

source (built from)	buildinfo (stamped into program)
directory (no git)	module (devel)
Go module	module v0.3.1-0.20260105212325-5347ac5f5bcb
directory (git)	module v0.0.0-20260131174001-ccb1d233f2a4+dirty vcs.revision=ccb1d233f2a43e9118b9146b3c9a5ded1efb7551 vcs.time=2026-01-31T17:40:01Z vcs.modified=true

► Go code to programmatically read the version

This is what it looks like in practice:


```
% go install github.com/gokrazy/tools/cmd/gok@latest
% gok --version
https://github.com/gokrazy/tools/commit/8ed49b4fafc7
```

But a version built from git has the full revision available (→ you can tell them apart):

```
% (cd ~gokrazy/./tools && go install ./cmd/...)
% gok --version
https://github.com/gokrazy/tools/commit/ba6a8936f4a88ddcf20a3b8f625e323e65664aa6
(modified)
```

VCS rev with NixOS

When packaging Go software with Nix, it's easy to lose Go VCS revision stamping:

1. Nix fetchers like `fetchFromGitHub` are implemented by fetching an archive (`.tar.gz`) file from GitHub — the full `.git` repository is not transferred, which is more efficient.
2. Even if a `.git` repository is present, Nix usually intentionally removes it for reproducibility: `.git` directories contain packed objects that change across `git gc` runs (for example), which would break reproducible builds (different hash for the same source).

So the fundamental tension here is between reproducibility and VCS stamping.

Luckily, there is a solution that works for both: I created the [stapelberg/nix/go-vcs-stamping](https://github.com/stapelberg/nix-go-vcs-stamping) [Nix overlay module](#) that you can import to get working Go VCS revision stamping by default for your `buildGoModule` Nix expressions!

.

The Nix Go build situation in detail

Tip: If you are not a Nix user, feel free to skip over this section. I included it in this article so that you have a full example of making VCS stamping work in the most complicated environments.

Packaging Go software in Nix is pleasantly straightforward.

For example, the Go Protobuf generator plugin `protoc-gen-go` is packaged in Nix with <30 lines: [official nixpkgs protoc-gen-go package.nix](#). You call [buildGoModule](#), supply as `src` the result from [fetchFromGitHub](#) and add a few lines of metadata.

But getting developer builds fully stamped is not straightforward at all!

When packaging my own software, I want to package individual revisions (developer builds), not just released versions. I use the same `buildGoModule`, or `buildGoLatestModule` if I need the latest Go version. Instead of using `fetchFromGitHub`, I provide my sources using Flakes, usually also from GitHub or from another Git repository. For example, I package `gokrazy/bull` like so:

```
{
  pkgs,
  pkgs-unstable,
  bullsrc,
  ...
}:

# Use buildGoLatestModule to build with Go 1.26
# even before NixOS 26.05 Yarara is released
# (NixOS 25.11 contains Go 1.25).
pkgs-unstable.buildGoLatestModule {
  pname = "bull";
  version = "unstable";

  src = bullsrc;

  # Needs changing whenever `go mod vendor` changes,
  # i.e. whenever go.mod is updated to use different versions.
  vendorHash = "sha256-sU5j2dji5bX2rp+qwwSFccXNpK2LCpWJq40mz/jmaXU=";
}
```

The `bullsrc` comes from my `flake.nix`:

► [Click here to expand the full `flake.nix`](#)

Go stamps all builds, but it does not have much to stamp here:

- We build from a directory, not a Go module, so the module version is `(devel)`.
- The stamped `buildinfo` does not contain any `vcs` information.

Here's a full example of `gokrazy/bull`:

```
% go version -m \
/nix/store/z3y90ck0fp1wwd4scljffhwxcrxjhb9j-bull-unstable/bin/bull
/nix/store/z3y90ck0fp1wwd4scljffhwxcrxjhb9j-bull-unstable/bin/bull: go1.26.1
      path    github.com/gokrazy/bull/cmd/bull
      mod     github.com/gokrazy/bull (devel)
      dep     github.com/BurntSushi/toml          v1.4.1-0.20240526193622-
a339e1f7089c
      dep     github.com/fsnotify/fsnotify        v1.8.0
      dep     github.com/google/renamio/v2         v2.0.2
      dep     github.com/yuin/goldmark             v1.7.8
      dep     go.abhg.dev/goldmark/wikilink        v0.5.0
      dep     golang.org/x/image                   v0.23.0
      dep     golang.org/x/sync                    v0.10.0
      dep     golang.org/x/sys                     v0.28.0
```

```
build    -buildmode=exe
build    -compiler=gc
build    -trimpath=true
build    CGO_ENABLED=0
build    GOARCH=amd64
build    GOOS=linux
build    GOAMD64=v1
```

To fix VCS stamping, add my `goVcsStamping` overlay to your `nixosSystem.modules`:

```
{
  nixpkgs.overlays = [
    stapelbergnix.overlays.goVcsStamping
  ];
}
```

(If you are using `nixpkgs-unstable`, like I am, you need to apply the overlay in both places.)

After rebuilding, your Go binaries should newly be stamped with `vcs` buildinfo:

```
% go version -m /nix/store/z8mgsf10pkc6dgvi8pfnbb7cs23pqfkn-bull-unstable/bin/bull
[...]
```

```
build    vcs=git
build    vcs.revision=c0134ef21d37e4ca8346bdcb7ce492954516aed5
build    vcs.time=2026-03-22T08:32:55Z
build    vcs.modified=false
```

Nice! 🥳 But... how does it work? When does it apply? How do you know how to fix your config?

I'll show you **the full diagram** first, and then explain how to read it:

.

There are 3 relevant parts of the Nix stack that you can end up in, depending on what you write into your `.nix` files:

1. Fetchers. These are what Flakes use, but also non-Flake use-cases.
2. Fixed-output derivations (FOD). This is how `pkgs.fetchgit` is implemented, but the constant hash churn (updating the `sha256` line) inherent to FODs is annoying.
3. Copiers. These just copy files into the Nix store and are not git-aware.

For the purpose of VCS revision stamping, you should:

- Avoid the Copiers! If you use Flakes:
 - ❌ do not use `url = "/home/michael/dcs"` as a Flake input
 - ✅ use `url = "git+file:///home/michael/dcs"` instead for git awareness
- I avoid the fixed-output derivation (FOD) as well.

- Fetching the git repository at build time is slow and inefficient.
- Enabling `leaveDotGit`, which is needed for VCS revision stamping with this approach, is even more inefficient because a new Git repository must be constructed deterministically to keep the FOD reproducible.

Hence, we will stick to the left-most column: fetchers.

Unfortunately, by default, with fetchers, the VCS revision information, which is stored in a Nix attrset (in-memory, during the build process), does not make it into the Nix store, hence, when the Nix derivation is evaluated and Go compiles the source code, Go does not see any VCS revision.

My [stapelberg/nix/go-vcs-stamping Nix overlay module](#) fixes this, and enabling the overlay is how you end up in the left-most lane of the above diagram: the happy path, where your Go binaries are now stamped!

My workaround: Nix git buildinfo overlay

How does the `go-vcs-stamping` overlay work? It functions as an adapter between Nix and Go:

- Nix tracks the VCS revision in the `.rev` in-memory attrset.
- Go expects to find the VCS revision in a `.git` repository, accessed via `.git/HEAD` file access and `git(1)` commands.

So the overlay implements 3 steps to get Go to stamp the correct info:

1. It synthesizes a `.git/HEAD` file so that Go's `vcs.FromDir()` detects a git repository.
2. It injects a `git` command into the `PATH` that implements exactly the two commands used by Go and fails loudly on anything else (in case Go updates its implementation).
3. It sets `-buildvcs=true` in the `GOFLAGS` environment variable.

For the full source, see [go-vcs-stamping.nix](#).

The clean fix

See [Go issue #77020](#) and [Go issue #64162](#) for a cleaner approach to fixing this gap: allowing package managers to invoke the Go tool with the correct VCS information injected.

This would allow Nix (or also gokrazy) to pass along buildinfo cleanly, without the need for [workarounds like my go-vcs-stamping adapter](#).

At the time of writing, issue #77020 does not seem to have much traction and is still open.

Conclusion: Stamp it! Plumb it! Report it!

My argument is simple:

Stamping the VCS revision is conceptually easy, but very important!

For example, if the production system from the incident I mentioned had reported its version, we would have saved multiple hours of mitigation time!

Unfortunately, many environments only identify the build output (useful, but orthogonal), but do not plumb the VCS revision (much more useful!), or at least not by default.

Your action plan to fix it is just 3 simple steps:

1. Stamp it! Include the source VCS revision in your programs.
 - This is not a new idea: [i3](#) builds include their [git-describe\(1\)](#) revision since 2012!
2. Plumb it! When building / packaging, ensure the VCS revision does not get lost.
 - My [“VCS rev with NixOS”](#) case study section above illustrates several reasons why the VCS rev could get lost, which paths can work and how to fix the missing plumbing.
3. Report it! Make your software print its VCS revision on every relevant surface, for example:
 - **Executable programs:** Report the VCS revision when run with `--version`
 - For Go programs, you can always use `go version -m`
 - **Services and batch jobs:** Include the VCS revision in the startup logs.
 - **Outgoing HTTP requests:** Include the VCS revision in the User-Agent
 - **HTTP responses:** Include the VCS revision in a header (internally)
 - **Remote Procedure Calls (RPCs):** Include the revision in RPC metadata
 - **User Interfaces:** Expose the revision somewhere visible for debugging.

Implementing “version observability” throughout your system is a one-day high-ROI project.

With my Nix example, you saw how the VCS revision is available throughout the stack, but can get lost in the middle. Hopefully my resources help you quickly fix your stack(s), too:

- [My stapelberg/nix/go-vcs-stamping overlay](#) for Nix / NixOS
- [My stampit repository](#) is a community resource to collect examples (as markdown content) and includes a Go module with a few helpers to make version reporting trivial.

Now go stamp your programs and data transfers! 🚀

Did you like this post? [Subscribe to this blog's RSS feed](#) to not miss any new posts!

I run a blog since 2005, spreading knowledge and experience for over 20 years! :)

If you want to support my work, you can [buy me a coffee](#).

Thank you for your support! ❤️